

skill:让 AI 输出规范易懂的人话

此 skill 是我个人整理的一套**写作规范**, 来自对真实 AI 输出中常见毛病的归纳, 用来把模型的输出改写成**规范易懂**的人话, 并能显著**减少 AI 味**。内容涵盖用词表述、行文组织、举例类比、排版等方面, 适用于 AI 讲解、教学、写作等场景。

此 skill 开源在 <https://github.com/Tsinswreng/skill-elucidate>。使用方式与一些注意要点在**文末**。本文先展示 skill 的正文内容, 正文内容如下:

1 | 使用规范的现代汉语书面语

1.1 | 表述要清晰

1.1.1 | 不要缺主语

错误示例:

✗ 点击保存按钮后, 会把修改保存到数据库。

正确示例:

✓ 用户点击保存按钮后, 系统会把修改保存到数据库。

1.1.2 | 不要缺谓语

错误示例:

✗ 此项目名叫 MyProj, 个人项目。

正确示例:

✓ 此项目名叫 MyProj, 是个人项目。

1.1.3 | 指代要明确

避免滥用「他」「其」「这」「前/后者」等代词, 尤其是在上下文中概念较多时。即使你做到了指代无歧义, 滥用代词依然会增大读者的阅读负担。应指明对象。

错误示例:

✗ 服务器收到请求后把它交给后端, 后端处理它, 再把它返回给前端。

正确示例:

✓ 服务器收到请求后把**请求**转交给后端, 后端处理**该请求**, 再把**处理结果**返回给前端。

指代对象要指名名词, 读者不用回头猜「它」是谁。

1.1.4 | 说清限定语/说清事物的所属

例:

1. 错误示例:

✗ 并发量增加之后响应时间显著变长, 瓶颈大概率出在数据库连接池不够用, 建议先把上限调大再看。

上限到底是甚么上限?

正确示例:

✓ 并发量增加之后响应时间显著变长, 瓶颈大概率出在数据库连接池不够用, 建议先把**数据库连接池的**上限调大再看。

2. 错误示例:

✗ 攻击工作由大模型驱动, **下一条命令**不可预先审查。

唐突冒出「下一条命令」,让人不知所指

正确示例:

✓ 攻击工作由大模型驱动: 攻击命令是大模型现场生成的, 下一条命令在事前并不存在, 因此无法预先审查。

1.1.5 | 说清动作对象

如 回复、传达 等词, 说清是谁给谁。

1.1.6 | 不使用不明确的缩写/代号等

错误示例:

✗ MyProj 是某某比赛一等奖项目 (1/114514) 。

正确示例:

✓ MyProj 是某某比赛一等奖项目, 在 114514 个参赛项目中排名第一。

1.2 | 规范用词

1.2.1 | 不要强行把双字词砍成单字

错误示例:

✗ 包间互引用 `workspace:*` 语法——引同仓成员、版本跟着走、不从 npm 拉。

正确示例:

✓ 包之间互相引用用的是 `workspace:*` 语法, 引用的就是同一仓库里的其他成员包, 依赖版本由仓库内该包的当前版本决定, 不需要从 npm 拉取。

1.2.2 | 避免自造抽象术语

错误示例:

✗ 此模块管容器的生灭

正确示例:

✓ 此模块管理容器的创建与销毁

1.2.3 | 除非术语或专门称呼, 避免夹杂英文

错误示例:

✗ 调用方只依赖抽象, 不 care 内部实现细节

正确示例:

✓ 调用方只依赖抽象, 不关心内部实现细节

1.3 | 避免过度口语化

文档正文要使用书面语, 不要把日常交谈的口吻写进正文。 错误示例:

✗ 这玩意负责把数据搞到数据库里去。

正确示例:

✓ 此组件负责把数据写入数据库。

1.4 | 避免凭空否定

先直接看例子:

问: 你是哪国人?

错误示例:

✗ 答: 我**不是**日本人, **不是**韩国人, **而是**中国人。

正确示例:

✓ 答: 我是中国人。

让你介绍一样东西时 你就直接说「他是甚么」/「他怎么样」, 若没人提到 xxx 你就不要凭空否定, 不要说「他不是 xxx 而是 yyy」。

仅在需要澄清真实的误解或混淆项时才使用, 禁止做无意义的[凭空捏造出一样东西再去否定]。

判别标准: 被否定的那个理解必须是已经出现过的, 或确确实实容易引起混淆的。

例:

1.
 - 问: 我看你长得像东亚人, 你是日本人吗?

- ☒ 答: 我不是日本人。我是中国人

2.

- 问: 软删除是甚么
- 答: ☒ 软删除是把数据中的记录标记为已删除, 而不会真的物理删数据的记录 ☒ 先说是甚么, 再否认易误解项
- 答: ☒ 软删除不会真的物理删数据的记录, 而是把数据中的记录标记为已删除 ☒ 问甚么就应该先答甚么

1.5 | 禁止使用破折号

1.6 | 禁止滥用引号或括号作解释说明

2 | 行文

2.1 | 注重逻辑连贯, 注重过渡

- 句子之间、段落之间的因果、顺序、转折关系要写明, 不要让读者脑补。
- 用「先...再...」「若...则...」「因此」「另一方面」等连接词点明关系。

错误示例:

✗ 服务器收到请求。缓存命中就返回。没命中就查数据库。数据库也没有就返回 404。

正确示例:

✓ 服务器收到请求后先查缓存。若缓存命中, 则直接返回缓存结果; 若未命中, 则再查数据库, 若数据库中也没有则返回 404。

2.2 | 明确中心主旨/主题/写作目的

文章逻辑要连贯, 衔接自然, 文章的每一部分都应为中心主旨服务, 和写作目的/写作主题有关联。不能想到甚么就说甚么。不要让用户疑惑「我明明在看关于 xxx 的文章, 现在却看的是 yyy 的内容, 你写的这些 yyy 和 xxx 有甚么关系」

常见的病是相关内容放错位置: 开头向读者许诺一条主线, 中途整段铺开通用背景, 进出都没有交代, 读者一到那里就迷路。

错误示例:

✗ 本文跟着一个网页请求走一遍: 从你按下回车, 到页面显示在眼前。你在地址栏输入 `example.com`, 按下回车。浏览器不知道这个名字对应哪台服务器, 第一件事是查它的地址。

DNS 诞生于 1983 年, 设计目的是取代手工维护的主机表。DNS 采用分层结构, 根域、顶级域、权威域逐级授权。查询方式分两种: 递归查询把问题整个交给服务器代办; 迭代查询则是逐层问路, 答案自己汇总。记录类型也多样, A 记录、CNAME 记录、MX 记录各有用途。

查到地址后, 浏览器与服务器建立连接。

读者读到中段便会问: 「我明明在看这次请求的旅程, 现在读的是 DNS 的通用介绍, 这些内容和这次请求有甚么关系?」

正确示例:

✓ 本文跟着一个网页请求走一遍: 从你按下回车, 到页面显示在眼前。你在地址栏输入 `example.com`, 按下回车。浏览器不知道 `example.com` 的服务器在哪, 第一件事是查它的地址, 这一步叫域名解析。浏览器先翻自己的缓存, 缓存里没有, 就把名字交给本地 DNS 服务器; 本地服务器也不知道, 就

逐层向上问,直到有服务器给出答案。地址到手,浏览器与服务器建立连接。

同样讲了 DNS,由「浏览器不知道服务器在哪」把它带出来,只讲这次请求会用到的部分,讲完用「地址到手」送回主线。

3 | 多举例子 结合例子说明

示例:

问: 等差数列怎么求和?

错误示例:

✗ 答: 首项加末项乘项数除以 2。

正确示例:

✓ 答: 首项加末项乘项数除以 2。 例如 2, 4, 6, 8, 10 这个等差数列, 该数列的首项为 2, 末项为 10, 因为一共有 5 项故项数为 5, 故该数列的和为:

$$(2+10)*5/2=30$$

4 | 类比与比喻

- 仅在适当的时间使用, 不要滥用。
- 滥用的比喻、不恰当的比喻增加理解成本、降低深度。
- 类比要恰当, 多用同类的事物来类比, 少用抽象的比喻。

错误示例:

✗ 垃圾回收就像一个勤劳的清洁工每天打扫卫生。

正确例:

✓ C# 的 `Task` 就相当于 js 的 `Promise` ✓ 恰当的类比

5 | 排版

文章除了内容上要易懂之外, 在排版上也要下功夫, 让人看得舒服。

5.1 | 注意分行分段

该分行分段就分, 避免一路小句写到尾。

错误示例:

✗ 缓存用来减少数据库的压力。读取数据时先查缓存,命中就直接返回;未命中才查数据库,并把结果写回缓存。缓存也有代价。它占用内存,而且数据可能过期,所以要设置失效策略,否则会读到旧值。

正确示例:

✓ 缓存用来减少数据库的压力。读取数据时先查缓存,若命中就直接返回;未命中才查数据库,并把结果写回缓存。

缓存也有代价。它占用内存,而且数据可能过期,所以要设置失效策略,否则会读到旧值。

5.2 | 善用列表与加粗

可采用列表、加粗等语法让文章更有层次

错误示例:

✗ 其一, xxx; 其二, yyy

正确示例:



1. xxx
2. yyy

5.3 | 引文 要写明 突出

错误示例:

- ✗ 快照的内容大意是: 项目名: 「juice-shop」; 零个攻击手在运行; 并发上限是七; 可用的攻击手提示词只有一份, 名字叫 tsinswreng-security 模型能看见的只有这段发过去的文字, 系统内部存了什么数据, 模型一无所知。

正确示例:

- ✓ 快照的内容大意是:
 - 项目名: 「juice-shop」;
 - 正在运行的攻击手: 0 个;
 - 并发上限: 7;
 - 可用的攻击手提示词: 只有一份, 名字叫 tsinswreng-security

模型能看见的只有这段发过去的文字， 系统内部存了什么数据， 模型一无所知。

注: 如果你使用 typst, 在 typst 中, 引用格式语法为:

```
#quote(block:true)[
  引文内容
]
```

typst

6 | 综合示例

1. 错误示例

✗ 此工作区是 MyProj——面向某某场景的多 Agent 协作平台, Bun monorepo。 某某比赛一等奖项目 (1/114514)。 **公司项目, 双仓库:** GitHub 开源侧 `company-org/MyProj` (公司 org); 公司内部侧 `ComMyProj` (gitlab 托管、非 GitHub, 历史与开源侧完全独立, 最新改动都在此。 用户是公司员工、接手负责开发)。 **主开发场 (2026-09-18 起):** 工作区内 ComMyProj 的 `dev` 分支

正确示例

✓ 此工作区属于 MyProj 项目。MyProj 是面向某某场景的多 Agent 协作平台，采用 Bun 单体仓库结构。MyProj 在某某比赛中获得一等奖，在 114514 个参赛项目中排名第一。

MyProj 是公司项目，对应两个远端仓库：

- GitHub 开源侧的仓库名为 `company-org/MyProj`，为公司的开源仓库；
- 公司内部侧的仓库名为 `ComMyProj`，它托管在公司内部的 gitlab 上；与 GitHub 开源侧没有共同的提交历史。用户是公司员工，接手负责开发。

公司的最新改动都在公司内部侧仓库，因此从 2026-09-18 起，主开发场是工作区内 ComMyProj 的 `dev` 分支。

2. 错误示例

✗ 起容器时任务书随启动命令送进去。之后容器里的每一步经 rpc 实时流回宿主。为甚么分这两层：动手的活是模型驱动的，下一条命令不可预先审查，不能让它直接在宿主上跑；一个攻击手一个容器，谁

把环境打挂了不牵连别人和宿主；环境一次性，装过什么、留下什么垃圾都随容器消失。

正确示例

✓ 容器启动时,任务书随启动命令送进容器。之后容器里的每一步都经由 rpc 实时流回宿主。为什么要分成宿主进程与容器两层: 因为攻击工作由大模型驱动, 攻击命令由大模型当场生成, 大模型输出的下一条命令无法预先审查, 难免混有破坏性的操作, 所以这样的命令不能直接在宿主上运行。一个攻击手配一个容器。即使一个容器里的环境损坏, 也不会牵连其他容器和宿主。环境是一次性的。容器里安装过什么、留下什么垃圾, 都随着容器一起消失。

3. 错误示例

✗ 派工决策到了 `runtime` 手里才落地: 构建镜像、起容器、回收容器。一个攻击手一个容器, 一次任务一个隔离沙箱: 打挂互不影响, 环境用完即抛。起容器当场, 宿主把任务书拼好、随启动命令送进去。任务书是中文模板, 拼五样: 项目名与描述; 目标入口清单,

每条带指纹；情报池两块看板与已交报告的三段摘要；行为要求清单；最后把开工便条以「最高优先级遵循」附在末尾。

正确示例

✓ 派工决策由 `runtime` 执行, `runtime` 负责做三件事: 构建镜像、创建容器、回收容器。一个攻击手配一个容器, 一次任务就是一个隔离沙箱: 各容器的环境损坏互不影响, 环境用完随容器一起丢弃。容器启动时, 宿主把组装好的任务书随启动命令送进容器。任务书是中文模板, 由五部分组成:

1. 项目名与描述
2. 目标入口清单, 每条带有指纹
3. 情报池两块看板与已交报告的三段摘要
4. 行为要求清单
5. 开工便条, 附在任务书末尾, 标着最高优先级

7 | 重构表述

重构表述 即 把有问题的文本改写成表述清晰的文本。

重构表述的要点:

- 逐条检查是否满足上述要求
- 站在人类读者的角度审视是否易懂? 是否有问题?
- 重构后的文本是否缺少必要的信息量?

7.1 | 重构不应减少有效信息量

重构表述只是换一种表述方法, 重构后该有的有用信息量应该保持不变。
对于原文中不必要的冗余的内容, 才可以考虑删除。

8 | 自查

问自己两个问题:

- 是否满足了上述要求:
- 站在人类读者的角度 是否易懂? 是否有问题?

-----skill 正文到此结束-----

1 | 安装

找到 github 仓库中的 `SKILL.md`,

- 简中: <https://github.com/Tsinswreng/skill-elucidate/blob/master/Langs/zh-Hans/tsinswreng-elucidate/SKILL.md>
- 繁中: <https://github.com/Tsinswreng/skill-elucidate/blob/master/Langs/zh-Hant/tsinswreng-elucidate/SKILL.md>

手动放到项目的 skill 目录下(如 `.agent/skills/tsinswreng-elucidate/`)。该 skill 的默认名称是 `tsinswreng-elucidate` , 其中 `tsinswreng-` 是本人名称, 充当命名空间, 无实际意义。

您可自行按需更改该 skill 任何部分的内容, 不限于名称、描述、正文等, 反正此 skill 就是一段自然语言文本。

2 | 使用例

这里展示一个[使用该 skill 来润色文本]的实例。使用的模型是 glm-5.2-highspeed。以下是未使用此 skill 时模型输出的一段文字, 来自在某项目中 AI 对遇到的一个 bug 的总结, 内容完全由模型自己发挥:

OpenCode 版本必须精确 pin(当前 1.18.23)并端到端验证: Docker Hub 的 `openeuler/opencode` 标签和 npm 的 `opencode-ai` 是两套版本; 历史上“版本

不低于下限”的策略造成开发/生产漂移, token 统计全坏。禁止用可变 `latest` 。

可见无约束时, AI 直接输出的内容非常符合 AI 的文风, 且看起来不是这么清晰易懂。随后要求大模型按要求改写润色, 人工发送以下提示词给大模型:

看 elucidate skill, 改写这段话。

改写后的一次产出:

OpenCode 的版本必须固定到具体的版本号, 当前固定的版本是 1.18.23; 固定之后还要对版本做端到端验证。这条规定的依据如下:

1. Docker Hub 上 `openeuler/opencode` 镜像的标签与 npm 上 `opencode-ai` 包的版本号是两套互不相干的版本体系, 确认版本时必须两边都查。
2. 历史上采用过「版本不低于下限」的策略, 也就是只要求版本号达到某个下限, 不锁定具体版本。这个策略造成开发环境与生产环境所用的版本发生漂移, 最终 token 统计功能完全损坏。
3. `latest` 是可变标签, `latest` 指向的版本会随时间变化, 因此禁止使用 `latest` 。

可见, 该 skill 还是有一定效果的。

3 | 几个注意点

1. 建议先让 AI 直接输出, 再让 AI 按照 skill 修改。相较于直接让 AI 按照 skill 规则一次性输出完, 这样做的话, 「初始输出」与「按 skill 修改润色」被人为分成了两个任务, 效果往往会更好。

原因在于模型的注意力有限。生成时, 模型的注意力要分配到上下文的全部内容上, 上下文里的规约越多, 每条规约分到的注意力越少, 模型越容易漏掉上下文中间的内容。所以分两遍做: 第一遍让模型写出初稿, 第二遍让模型拿规约逐条对照初稿修改。这样第二遍只有对照与修改一件事, 每条规约才更容易被遵守执行。

2. 此 skill 不是银弹, 产出的文本的品质模型能力有关。一次性修改未必能使您完全满意, 可适当放低期待, 让 AI 按 skill 多轮对照再修改, 不必指望 AI 能一次性改好。通常每轮润色修改后的产出品质都会有提升。